“Question Bank Management System”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

MANASWI	**: 2520030411**
NITHYA	**: 2520030517**
SRUTHIKA	**: 2520030361**

Team Number: 23

Section: 2

Under the Guidance of

Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Question Bank Management System"** is a record of Bonafide work of **MANASWI– 25200300411, NITHYA – 2520030517 , SRUTHIKA – 2520030361** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

MANASWI – 2520030411
NITHYA – 2520030517
SRUTHIKA – 2520030361

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"Question Bank Management System"** is a bonafide work done and submitted by **MANASWI - 2520030411, NITHYA – 2520030517 SRUTHIKA – 2520030361** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD
(Dr. P. Venkateshwara Rao)

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his valuable guidance, constant encouragement and timely suggestions throughout the course of this project. His insights into data structures and algorithms helped us to choose the right approach for every module of the Question Bank Management System.

We are thankful to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the facilities and a supportive environment to carry out this work. We also thank all the faculty members of the department for their help and cooperation.

Finally, we thank our friends and family members for their continuous support and motivation, which helped us to complete this project successfully.

MANASWI – 2520030411
NITHYA – 2520030517
SRUTHIKA - 2520030361

Abstract

The Question Bank Management System is a web-based platform designed to store, organise and reuse examination questions in one central repository. Faculty members add questions together with their subject, unit, topic, marks, difficulty level, cognitive level and answer, and submit them for review. Approved questions become part of the question bank, from which faculty can search and filter questions quickly and generate balanced question papers according to a blueprint, while students can take topic-wise practice tests. The system reduces repeated effort, avoids duplicate and over-used questions and keeps the quality of question papers consistent.

A review workflow lets administrators approve or reject the questions submitted by faculty, and an administrative module manages users, subjects, topics and reports. By bringing question setting, review, storage and paper generation together in one place, the Question Bank Management System saves the time of faculty, improves the fairness of examinations and helps the institution maintain an organised and reusable question repository. The system applies suitable data structures and algorithms, namely hash maps, a trie, an inverted index, a syllabus tree, a prerequisite graph, heaps, queues and stacks, so that searching, duplicate detection, paper generation and reviewing remain efficient as the number of questions grows.

Keywords: Question bank, Question paper generation, Hash map, Trie, Inverted index, Graph, Priority queue, Web application.

Table of Contents

			Page No.
	Acknowledgement		
	Abstract		
	Table of Contents		
	List of Figures		
1	Introduction	1.1 Overview 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Organization of the Report	1–4
2	System Requirements	2.1 Existing Systems and Literature Review 2.2 Functional Requirements 2.3 Non-Functional Requirements 2.4 Hardware Requirements 2.5 Software Requirements	5–8
3	System Design	3.1 High-level Architecture Diagram 3.2 Description of Each Layer 3.3 Module Design 3.4 Data Structures and Algorithms Used 3.5 Data Design 3.6 Workflow of Question Paper Generation	9–14
4	Implementation	4.1 Module-wise Implementation 4.2 Front-end Logic (UI Rendering and State Management) 4.3 API Integration 4.4 Persistence and Index Rebuilding	15–21
5	Results	5.1 Output Screens 5.2 Functional Test Cases 5.3 Complexity Analysis 5.4 Discussion	22–25
6	Challenges & Limitations	6.1 Issues Faced during Development 6.2 Solutions Applied 6.3 Limitations of the System	26–27
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills Learned during Development	28–29
	References	Books, tutorials, APIs and documentation sites used	30

List of Figures

1.1 Overview of the Question Bank Management System ecosystem 1

2.1 Use-case diagram of the Question Bank Management System 7

3.1 High-level architecture of the Question Bank Management System 9

3.2 Trie used for topic and keyword search with autocomplete 12

3.3 Topic prerequisite graph (DAG) with adjacency list 13

3.4 Flow chart of the question paper generation process 14

5.1 Login and registration page 22

5.2 Add question page 22

5.3 Question search with filters and autocomplete 22

5.4 Review queue and question approval page 23

5.5 Paper blueprint and generated question paper 23

5.6 Administrator dashboard 23

1. INTRODUCTION

1.1 Overview

Every examination begins with a set of good questions, yet in most institutions these questions are scattered across personal files, old question papers, e-mails and notebooks. Faculty members spend hours rewriting questions that already exist, the same questions are repeated from one examination to the next, and there is no easy way to check whether a paper covers all the units with the right mix of easy, medium and difficult questions. At the same time, institutions need well-organised question repositories for internal tests, university examinations, accreditation and student practice. Because the questions are kept informally, quality control, reuse and fair coverage of the syllabus are hard to achieve.

Question Bank Management System is a web-based platform designed to store, organise, review and reuse examination questions, as shown in Fig. 1.1. Faculty add questions with their subject, unit, topic, marks, difficulty and cognitive level, and submit them for review. Administrators approve the questions that meet the quality standard. Faculty then search the approved questions using filters and autocomplete and generate balanced question papers from a blueprint, while students attempt topic-wise practice tests. An administrative module manages users, subjects, topics, reviews and reports.

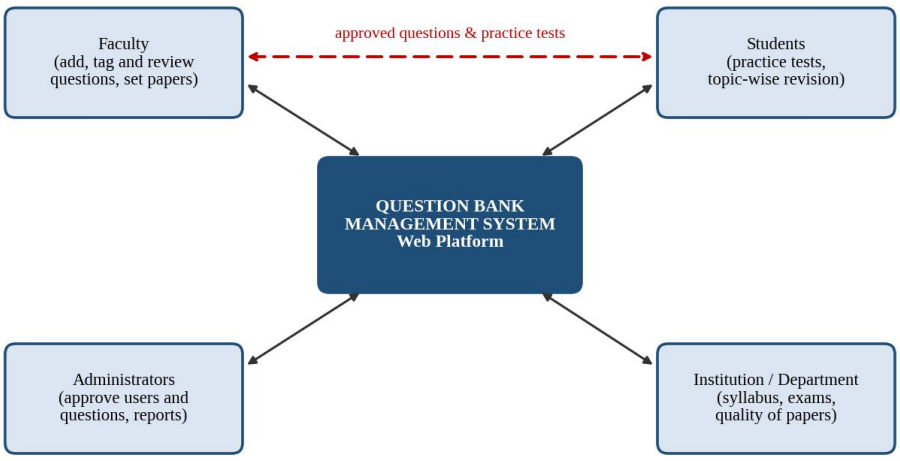


Fig. 1.1 Overview of the Question Bank Management System ecosystem

Because this project is carried out for the course Data Structures and

Algorithms-3, the emphasis is on how a careful choice of data structures and algorithms keeps such a system fast and scalable. A data structure is a way to organise and store data that is usually chosen for efficient access to data [1]. The Question Bank Management System uses hash maps for constant-time look-ups [2], a trie for prefix-based topic search [5], an inverted index for filters, a tree for the syllabus, a directed graph for topic prerequisites [4], [13], heaps to select and rank questions [1], a queue for the review workflow [3], and a stack to undo administrative actions.

1.2 Problem Statement

Faculty who prepare question papers usually cannot find suitable earlier questions quickly, and the institution has no reliable way to control the quality, difficulty and reuse of questions. The existing practice suffers from the following drawbacks:

- There is no centralised, searchable repository of questions classified by subject, unit, topic, marks and difficulty.
- There is no structured review process, so questions of doubtful quality or with wrong answers can enter question papers.
- Questions are repeated across tests and years, and duplicate or near-duplicate questions are hard to detect.
- Question papers are prepared manually, so the coverage of the syllabus and the balance of marks and difficulty depend on the effort of the individual setter.
- Students have no organised source of topic-wise practice questions, and the institution cannot easily measure how the questions are used.

The problem addressed in this work is to design and develop a web-based system in which faculty can add, tag, search and reuse questions, administrators can review and approve them, papers can be generated automatically from a blueprint, and students can practise by topic. The system must use appropriate data structures and algorithms so that response time remains low as the number of questions grows.

Several challenges are involved: keeping search responsive as questions accumulate, detecting duplicate questions, selecting questions that satisfy a

blueprint without over-using the same questions, keeping answers confidential, and keeping the in-memory indexes consistent with the data stored permanently in the database.

1.3 Scope of the Project

The scope of the Question Bank Management System covers the following functions:

- Registration, login and role-based access for faculty, students and administrators.
- Creation and maintenance of subjects, units and topics (the syllabus structure).
- Adding, editing and deleting questions with marks, difficulty, cognitive level and answer, with detection of duplicates.
- Search and filtering of questions by keyword, subject, topic, difficulty, marks and type.
- Review and approval of the questions submitted by faculty.
- Automatic generation of question papers from a blueprint, together with an answer key.
- Topic-wise practice tests for students, and an administrative module for users, reports and activity logs.

The following are outside the scope of the present version: online proctored examinations, automatic evaluation of descriptive answers, plagiarism checks against the internet, a native mobile application, and the use of machine learning to predict the difficulty of questions.

1.4 Objectives

- To provide one centralised repository for storing and organising examination questions.
- To let faculty add questions with complete details and find suitable questions quickly through search and filters.
- To maintain the quality of questions through a review and approval workflow.

- To generate balanced question papers automatically according to a blueprint of units, marks and difficulty.
- To avoid duplicate and over-used questions and to provide topic-wise practice tests for students.
- To apply hash maps, tries, trees, graphs, heaps, queues and stacks so that every major operation is efficient.
- To help the institution maintain an organised, reusable and secure question repository.

1.5 Organization of the Report

The remainder of this report is organised as follows. Chapter 2 presents the existing approaches, the functional and non-functional requirements, and the hardware and software specifications. Chapter 3 describes the system architecture, the modules and the choice of data structures. Chapter 4 explains the implementation of each module using pseudocode. Chapter 5 presents the results, including the output screens, test cases and complexity analysis. Chapter 6 discusses the challenges faced and the limitations of the system. Chapter 7 concludes the report and suggests future work. The references are listed at the end.

2. SYSTEM REQUIREMENTS

2.1 Existing Systems and Literature Review

Question banks and question papers are currently prepared by three broad kinds of methods. Manual preparation in documents and notebooks is the most common method, but it is slow, hard to search and difficult to share. Spreadsheets and shared folders give some structure, yet they have no review process, no duplicate detection and no support for paper generation. Commercial learning management and online examination platforms offer question banks, but they are usually general-purpose, costly and not aligned with the syllabus and policies of a particular institution. Table 2.1 summarises these approaches and the gaps that the Question Bank Management System aims to fill.

Table 2.1: Comparison of existing approaches

Approach	Strengths	Gaps / limitations
Manual documents and notebooks	No set-up cost; familiar to faculty	Hard to search; questions repeated; no review; papers prepared by hand
Spreadsheets and shared folders	Simple structure; easy to share	No duplicate detection or access control; no automatic paper generation
Commercial LMS / online exam platforms	Rich features; large question stores	General-purpose and costly; weak syllabus mapping; limited institutional control
Question Bank Management System (proposed)	Institution-managed, searchable, reviewed and reusable; automatic paper generation	Needs initial entry of questions and administrator review

Research on graph algorithms shows that directed acyclic graphs and topological ordering describe prerequisite relations between topics effectively [13], which motivates the use of a graph for the syllabus. The trie, introduced by Fredkin [5], is a classical structure for prefix search and autocomplete, heaps provide efficient retrieval of the best-ranked items [1], and the resemblance of two texts can be measured by comparing their sets of words [15], which is the basis of the duplicate check. Questions are classified by cognitive level using Bloom's taxonomy [14]. Existing solutions rarely combine these techniques in an institution-focused question bank, which is the gap addressed by this project.

2.2 Functional Requirements

The functional requirements describe the functions that the system must perform. They are grouped by module in Table 2.2, and the main interactions of each actor are shown in the use-case diagram of Fig. 2.1.

Table 2.2: Functional requirements

ID	Module	Requirement
FR1	Authentication	The system shall allow faculty and students to register with a valid e-mail and password and to log in and log out securely.
FR2	Authentication	The system shall provide role-based access for Faculty, Student and Administrator.
FR3	Subjects and topics	Administrators shall create subjects, units and topics and define the prerequisite relations between topics.
FR4	Subjects and topics	The system shall show the syllabus as a tree of subject, unit and topic and reject a prerequisite that creates a cycle.
FR5	Questions	Faculty shall add questions with subject, topic, marks, difficulty, cognitive level, type and answer.
FR6	Questions	Faculty shall edit and delete their own questions, and the system shall warn about duplicate or near-duplicate questions.
FR7	Search	Users shall search questions by keyword, subject, topic, difficulty, marks or type, with filters.
FR8	Search	The system shall provide autocomplete suggestions while a topic or keyword is being typed.
FR9	Review	Submitted questions shall wait in a review queue and be reviewed in the order of submission.
FR10	Review	Administrators shall approve or reject a question with a comment; only approved questions can be used in papers.
FR11	Paper generation	Faculty shall define a blueprint (units, marks, difficulty and number of questions) for a paper.
FR12	Paper generation	The system shall generate a paper that satisfies the blueprint, avoiding recently used questions, together with an answer key.
FR13	Practice	Students shall take topic-wise practice tests drawn from approved questions marked for practice.
FR14	Practice	The system shall suggest the order of study of topics using the prerequisite relations.
FR15	Administration	Administrators shall approve, block or delete users and moderate questions.
FR16	Administration	The system shall maintain an activity log and allow the last administrative action to be undone.
FR17	Reports	The system shall report the number of questions per topic and difficulty and the most and least used questions.

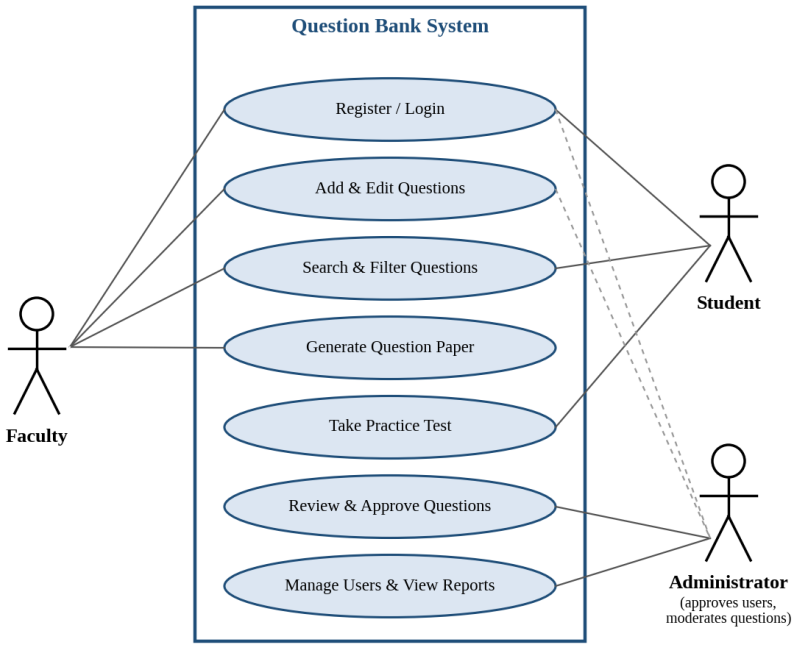


Fig. 2.1 Use-case diagram of the Question Bank Management System

2.3 Non-Functional Requirements

Non-functional requirements describe the quality attributes of the system, such as response time, usability, scalability and reliability. They are summarised in Table 2.3.

Table 2.3: Non-functional requirements

Attribute	Requirement
Performance / response time	Search results and autocomplete shall be returned within 2 seconds under normal load; a question paper shall be generated within 5 seconds; ordinary pages shall load within 3 seconds.
Usability	The interface shall be simple and consistent, with clear navigation, form validation and helpful error messages.
Scalability	The design shall support growth in the number of questions without redesign; core operations shall run in constant, logarithmic or near-linear time.
Reliability and availability	Data shall be stored permanently so that it is not lost on restart; indexes shall be rebuilt automatically at start-up.
Security	Passwords shall be stored only as salted hashes; all pages shall check the role of the user; inputs shall be validated to prevent injection.
Confidentiality	Answers and unused paper questions shall be visible only to faculty and administrators; students see answers only after a practice test.

Attribute	Requirement
Maintainability	The code shall be modular, with each module (question, search, paper, review, etc.) separately testable.
Portability / compatibility	The application shall work on current versions of Chrome, Edge and Firefox on desktop and mobile screens.

2.4 Hardware Requirements

Table 2.4: Hardware requirements

Component	Development machine	Deployment server
Processor	Intel Core i3 / AMD Ryzen 3 or better	2 vCPU or better
Memory (RAM)	4 GB minimum (8 GB recommended)	4 GB or more
Storage	500 MB free space	10 GB or more
Network	Broadband internet connection	Public IP / domain name
Display	1366 x 768 or higher	Not required

2.5 Software Requirements

Table 2.5: Software requirements

Category	Specification
Operating system	Windows 10/11, Ubuntu 22.04 or macOS
Front end	HTML5, CSS3 and JavaScript (ES6) [11]
Back end	Python 3.10 or later with the Flask web framework [9], [10]
Database	SQLite 3 for development; a relational server database such as MySQL for deployment [12]
Web browser	Google Chrome, Microsoft Edge or Mozilla Firefox (latest versions)
Tools	Visual Studio Code, Git and GitHub, Postman for API testing

These tools were chosen because they are free, widely documented and well suited to a team project of this size.

3. SYSTEM DESIGN

3.1 High-level Architecture Diagram

The Question Bank Management System follows a layered architecture in which each layer has a clear responsibility and communicates only with the layer next to it [7]. This separation makes the system easier to understand, test and extend. The four layers are the presentation layer, the application (business logic) layer, the data-structure (index) layer and the persistence layer, as shown in Fig. 3.1.

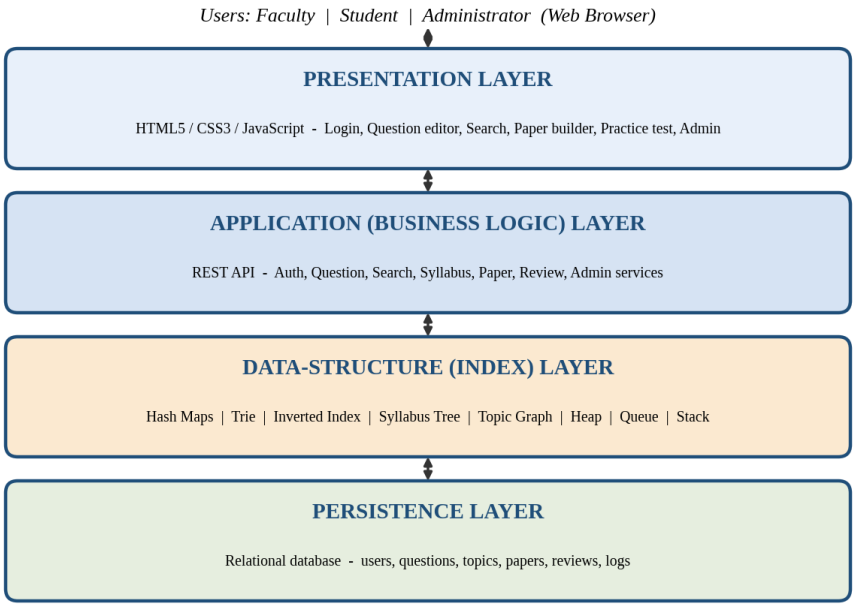


Fig. 3.1 High-level architecture of the Question Bank Management System

3.2 Description of Each Layer

3.2.1 Presentation layer

The presentation layer is the web interface that runs in the browser. It contains the login and registration pages, the question entry and editing forms, the search page with autocomplete, the review queue, the paper builder, the practice-test page and the administrator dashboard. It sends requests to the application layer and renders the responses, and it shows only the menus allowed for the role of the logged-in user.

3.2.2 Application (business logic) layer

The application layer exposes a set of REST-style services [8]: authentication, syllabus, question, search, review, paper generation, practice and administration. Each service validates its input, applies the business rules (for example, a question cannot be used in a paper until it is approved) and calls the data-structure layer for fast look-ups and ranking.

3.2.3 Data-structure (index) layer

This layer is the heart of the project. When the application starts, the data stored in the database is loaded into in-memory structures: hash maps for users, questions and topics, a trie and an inverted index for search, a tree and a directed graph for the syllabus, heaps for selection and ranking, a queue for the review workflow and a stack for undo. Because these structures live in memory, the most frequent operations avoid repeated database queries and run in the time complexities listed in Chapter 5.

3.2.4 Persistence layer

The persistence layer is a relational database that permanently stores users, subjects, topics, questions, reviews, papers, practice attempts and the activity log. Every change made through the application layer is written to the database as well as to the in-memory structures, so that the indexes can be rebuilt after a restart.

3.3 Module Design

The system is divided into six modules. Table 3.1 lists the responsibilities of each module and the main data structures it relies on.

Table 3.1: Modules and their responsibilities

Module	Responsibilities	Main data structures
User and Authentication	Registration, login, role-based access, approval of faculty accounts	Hash map (by id and by e-mail)
Syllabus and Topics	Subjects, units, topics, prerequisite relations, study order	Tree, directed graph, queue (topological sort)
Question Management	Add, edit and delete questions; duplicate detection; classification by difficulty and cognitive level	Hash map, hash set, inverted index
Search	Keyword, topic and filter search, autocomplete, ranking of results	Trie, inverted index, max-heap

Module	Responsibilities	Main data structures
Paper Generation and Practice	Blueprint-based paper generation, answer key, practice tests, spreading the use of questions	Min-heap, hash map, merge sort
Review and Administration	Review queue, approve or reject, user management, activity log with undo, reports	Queue (FIFO), stack, hash map

3.4 Data Structures and Algorithms Used

The choice of data structure for each requirement is justified in Table 3.2. In each case the structure was chosen because it gives the best time complexity for the operation that is performed most often.

Table 3.2: Mapping of requirements to data structures

Requirement	Data structure	Why it is suitable
Find a user or question by id or e-mail	Hash map	Average O(1) insertion and look-up [2]
Prefix search and autocomplete on topics and keywords	Trie	Cost depends on the length of the prefix, not on the number of questions [5]
Filter questions by subject, topic, difficulty or marks	Inverted index (hash map of hash sets)	Direct access to all questions with a given attribute; filters combine by set intersection
Show the syllabus as subject, unit and topic	Tree (n-ary)	Natural parent-child structure; traversal in O(number of nodes)
Order topics by prerequisites	Directed graph and topological sort	Gives a valid study order in O(V + E) and detects cycles [13]
Detect duplicate questions	Hash set and Jaccard similarity	Exact duplicates found in O(1); near-duplicates compared only with questions sharing words [15]
Select the least-used questions and the top-K matches	Min-heap / max-heap (priority queue)	Best item is obtained in O(log n) without sorting all candidates [1]
Review questions in the order of submission	Queue (FIFO)	Preserves the order of arrival
Undo the last admin action	Stack (LIFO)	The most recent action is the first to be reversed

3.4.1 Hash maps and inverted index

A hash map stores key–value pairs and gives average constant-time insertion and look-up [2]. The Question Bank Management System keeps one map

from e-mail to user (used at login), one from user id to user and one from question id to question. An inverted index maps every subject, topic, difficulty level and marks value to the set of question ids that have it, so a filter such as “topic = Stack and difficulty = Medium” returns its result by intersecting two sets without scanning all questions.

3.4.2 Trie

A trie is a tree in which every edge is labelled with a character and every path from the root spells a prefix [5]. To search for the prefix “so”, the system follows the edges s and o and then collects all words that end below that node, as illustrated in Fig. 3.2. The time taken depends only on the length of the prefix and the number of matches, so autocomplete stays fast even when thousands of topics and keywords are stored.

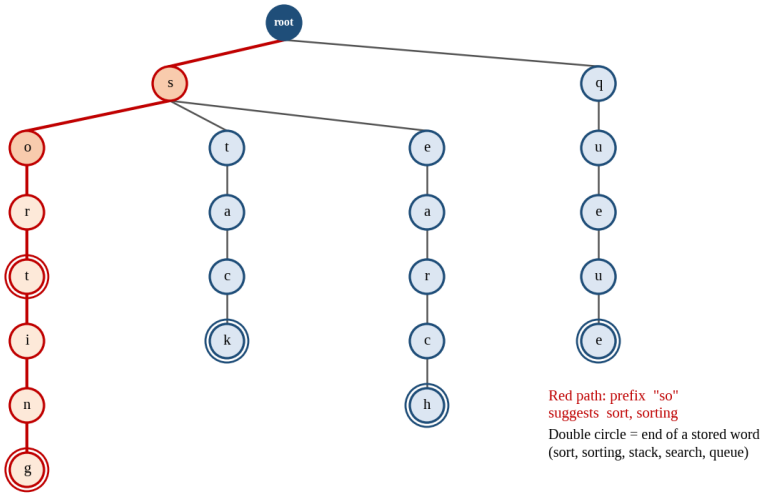


Fig. 3.2 Trie used for topic and keyword prefix search and autocomplete

3.4.3 Syllabus tree and prerequisite graph

The syllabus is stored as an n-ary tree whose root is the subject, whose children are the units and whose leaves are the topics. In addition, the topics are the vertices of a directed acyclic graph in which an edge from A to B means that topic A must be studied before topic B (Fig. 3.3). The graph is stored as an adjacency list, which needs memory proportional to the number of vertices plus edges [3], [4]. Before a new prerequisite is accepted, a depth-first search checks that it does not create a cycle, and Kahn’s algorithm [13] produces a valid study order for practice tests.

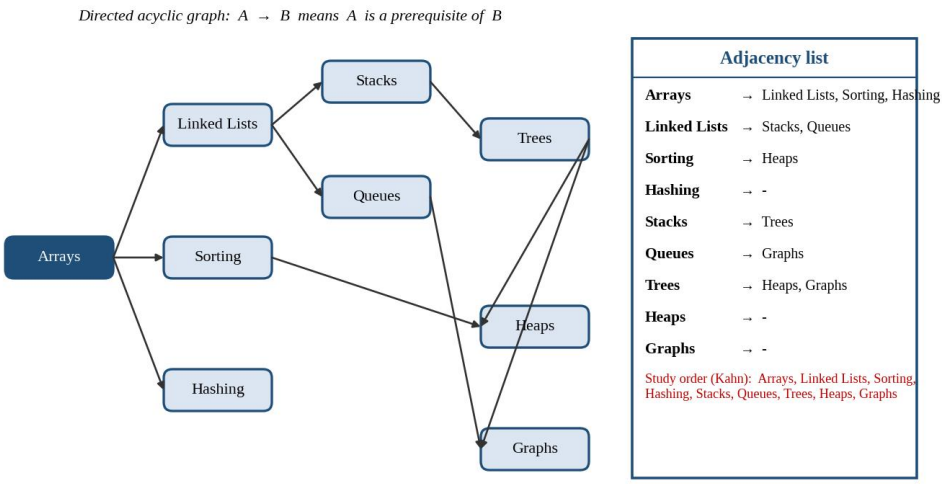


Fig. 3.3 Topic prerequisite graph (DAG) with adjacency list

3.4.4 Heap and sorting

A binary min-heap always keeps the item with the smallest key at the top, and a max-heap keeps the largest. When a paper section needs k questions out of n candidates, the system pushes the candidates into a min-heap keyed by usage count and removes the top k items, which selects the least-used questions without sorting all n items [1]. For lists that must be fully ordered, such as the questions of a paper by section and marks, merge sort is used because it is stable and runs in O(n log n) time [6].

3.4.5 Queue and stack

Submitted questions wait in a first-in-first-out queue so that they are reviewed in the order in which they were submitted. The administrator's activity log is kept as a stack so that the most recent action can be undone first.

3.5 Data Design

The permanent data of the system is organised into the entities listed in Table 3.3.

Table 3.3: Main entities and attributes

Entity	Key attributes
User	user_id, name, e-mail, password_hash, salt, role (FACULTY / STUDENT / ADMIN), status (PENDING / ACTIVE / BLOCKED)
Topic	topic_id, subject, unit, name, parent_id, prerequisites

Entity	Key attributes
Question	question_id, topic_id, text, type, marks, difficulty, bloom_level, answer, status (PENDING / APPROVED / REJECTED), usage_count, created_by
Review	review_id, question_id, reviewer_id, decision, comment, reviewed_at
Paper	paper_id, subject, total_marks, blueprint, created_by, created_at
PaperQuestion	paper_id, question_id, section, marks
ActivityLog	log_id, admin_id, action, target_id, timestamp

3.6 Workflow of Question Paper Generation

The generation of a question paper is the most algorithmic feature of the system. As shown in Fig. 3.4, the system reads the blueprint, which lists the unit, difficulty, marks and number of questions for every section. For each section, it intersects the inverted-index sets to find the approved questions that match, removes the questions used in recent papers, and pushes the remaining candidates into a min-heap keyed by usage count, from which the required number of questions is taken.

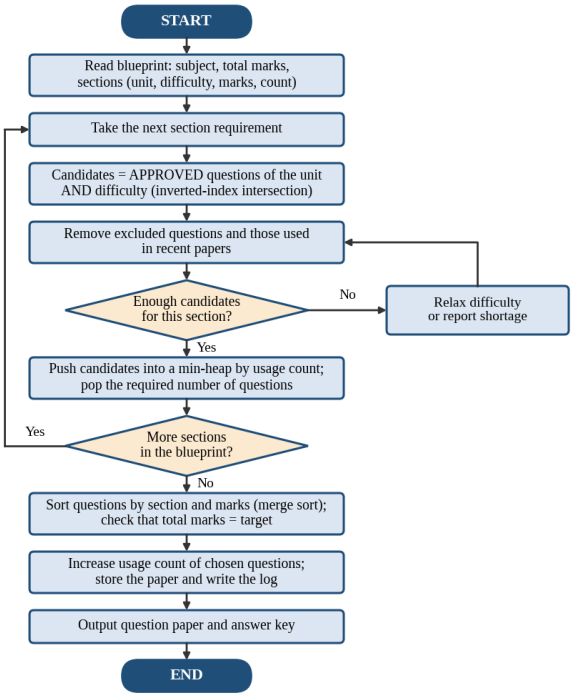


Fig. 3.4 Flow chart of the question paper generation process

This approach meets the blueprint exactly, spreads the use of questions evenly across papers and avoids sorting all the candidates. If a section has too few candidates, the system reports the shortage instead of failing silently.

4. IMPLEMENTATION

4.1 Module-wise Implementation

This section explains the implementation of each module using pseudocode. The pseudocode is independent of the programming language and uses the following conventions: “<-” denotes assignment, “//” starts a comment, and map.get, map.put and set.add are the usual hash-map and hash-set operations.

4.1.1 User registration and authentication

Registration checks that the e-mail is new, validates the input, stores a salted hash of the password and inserts the user into two hash maps. New students are activated immediately, while new faculty accounts start with the status PENDING until an administrator approves them. Login performs a constant-time look-up by e-mail and compares hashes.

Algorithm 4.1: RegisterUser

```
ALGORITHM RegisterUser(email, password, name, role)
 1  IF usersByEmail.contains(email) THEN
 2      RETURN Error("Email already registered")
 3  validate(email, password)          // format, minimum length
 4  salt <- generateSalt()
 5  hash <- hashFunction(password + salt)
 6  id   <- nextUserId()
 7  status <- (role = STUDENT) ? ACTIVE : PENDING
 8  user <- User(id, email, hash, salt, name, role, status)
 9  usersByEmail.put(email, user)      // O(1) average
10  usersById.put(id, user)
11  persist(user)
12  RETURN id
```

Algorithm 4.2: Login

```
ALGORITHM Login(email, password)
 1  user <- usersByEmail.get(email)
 2  IF user = NULL OR user.status != ACTIVE THEN
 3      RETURN Error("Invalid credentials")
 4  IF hashFunction(password + user.salt) != user.hash THEN
 5      RETURN Error("Invalid credentials")
 6  token <- createSession(user.id)
 7  sessions.put(token, user.id)
 8  RETURN token
```

4.1.2 Question management and duplicate detection

Whenever a question is added, its text is normalised (lower case, punctuation removed) and its hash is looked up to catch exact duplicates. The words of the question are then compared, using the Jaccard similarity, only with the

questions that share at least one of its words, so that near-duplicates are reported without comparing against the whole bank. An accepted question is placed in the review queue; after approval it is inserted into the trie and the inverted indexes, so that search always reflects the latest data.

Algorithm 4.3: AddQuestion

```
ALGORITHM AddQuestion(facultyId, q)
 1  words <- tokenize(normalize(q.text))
 2  h <- hashFunction(join(words))
 3  IF hashIndex.contains(h) THEN
 4      RETURN Error("Duplicate question")
 5  similar <- {}
 6  FOR EACH w IN words DO
 7      similar <- similar UNION wordIndex[w]
 8  FOR EACH id IN similar DO
 9      IF jaccard(words, wordsOf(id)) >= 0.8 THEN
10          RETURN Warning("Similar to question " + id)
11  q.id <- nextQuestionId(); q.status <- PENDING
12  questionsById.put(q.id, q); hashIndex.add(h)
13  FOR EACH w IN words DO wordIndex[w].add(q.id)
14  reviewQueue.enqueue(q.id)           // FIFO
15  persist(q)
```

4.1.3 Search and ranking

The trie supports insertion and prefix search. Each topic name and keyword is stored in the trie together with the ids of the questions that carry it. The search algorithm first collects candidates from the topic and keyword tries, then applies the filters by intersecting the candidate set with the sets from the inverted indexes, and finally ranks the candidates with a max-heap. The relevance score gives the highest weight to an exact topic match, followed by keyword match, difficulty match and marks match.

Algorithm 4.4: Trie insertion and prefix search

```
ALGORITHM Trie.insert(word, id)
 1  node <- root
 2  FOR EACH ch IN word DO
 3      IF ch NOT IN node.children THEN
 4          node.children[ch] <- new TrieNode()
 5      node <- node.children[ch]
 6  node.isEnd <- TRUE
 7  node.ids.add(id)

ALGORITHM Trie.prefixSearch(prefix)
 1  node <- root
 2  FOR EACH ch IN prefix DO
 3      IF ch NOT IN node.children THEN RETURN {}
 4      node <- node.children[ch]
 5  RETURN collectAll(node)       // DFS over the subtree
```

Algorithm 4.5: SearchQuestions

```
ALGORITHM SearchQuestions(query, filters, K)
INPUT : query text, filters (subject, topic, difficulty), K
OUTPUT: top-K approved questions ranked by relevance
 1  q <- lowercase(query)
 2  cand <- trieTopic.prefixSearch(q) UNION
            trieKeyword.prefixSearch(q)
 3  FOR EACH f IN filters DO
 4      cand <- cand INTERSECT index[f.type].get(f.value)
 5  heap <- new MaxHeap()
 6  FOR EACH id IN cand DO
 7      IF questionsById.get(id).status = APPROVED THEN
 8          heap.push(relevance(id, q, filters), id)
 9  result <- []
10  WHILE NOT heap.isEmpty() AND size(result) < K DO
11      result.append(heap.pop())
12  RETURN result
```

4.1.4 Syllabus tree and prerequisite graph

Subjects, units and topics are stored as a tree. A prerequisite between two topics is added as a directed edge in the topic graph, but only after a depth-first search confirms that the new edge does not create a cycle. The study order of the topics is then obtained with Kahn’s algorithm, which repeatedly removes the topics that have no unmet prerequisite.

Algorithm 4.6: AddPrerequisite and Reachable

```
ALGORITHM AddPrerequisite(a, b)        // a is studied before b
 1  IF a = b OR reachable(b, a) THEN
 2      RETURN Error("Cycle detected")
 3  topicGraph.adj[a].add(b)
 4  indegree[b] <- indegree[b] + 1
 5  persist(edge(a, b))

ALGORITHM reachable(src, dst)          // iterative DFS
 1  stack <- [src]; seen <- {src}
 2  WHILE stack is not empty DO
 3      v <- stack.pop()
 4      IF v = dst THEN RETURN TRUE
 5      FOR EACH w IN topicGraph.adj[v] DO
 6          IF w NOT IN seen THEN seen.add(w); stack.push(w)
 7  RETURN FALSE
```

4.1.5 Study order, paper generation and practice tests

The study order of the topics is computed with Kahn’s algorithm, described in Section 3.4.3. Paper generation follows the workflow of Section 3.6: for each section of the blueprint, the least-used approved questions that satisfy the unit and the difficulty are taken from a min-heap. A practice test for a student takes questions topic by topic in the study order and, within a topic, starts with the easy questions.

Algorithm 4.7: StudyOrder (Kahn)

```
ALGORITHM StudyOrder(subject)
 1  queue <- []; order <- []
 2  FOR EACH t IN topicsOf(subject) DO
 3      deg[t] <- indegree[t]
 4      IF deg[t] = 0 THEN queue.enqueue(t)
 5  WHILE queue is not empty DO
 6      v <- queue.dequeue(); order.append(v)
 7      FOR EACH w IN topicGraph.adj[v] DO
 8          deg[w] <- deg[w] - 1
 9          IF deg[w] = 0 THEN queue.enqueue(w)
10  RETURN order        // prerequisites come first
```

Algorithm 4.8: GeneratePaper

```
ALGORITHM GeneratePaper(blueprint, recent)
 1  paper <- []
 2  FOR EACH sec IN blueprint.sections DO
 3      cand <- unitIndex[sec.unit] INTERSECT
                 levelIndex[sec.level] INTERSECT approvedSet
 4      heap <- new MinHeap()           // key = usage count
 5      FOR EACH id IN cand DO
 6          IF id NOT IN recent THEN heap.push(usage[id], id)
 7      IF heap.size() < sec.count THEN
 8          RETURN Error("Not enough questions: " + sec.unit)
 9      FOR i <- 1 TO sec.count DO
10          paper.append((sec, heap.pop()))
11  mergeSort(paper, BY section, marks)
12  FOR EACH (sec, id) IN paper DO RecordUsage(id)
13  persist(paper); RETURN paper
```

Algorithm 4.9: BuildPracticeTest

```
ALGORITHM BuildPracticeTest(subject, n)
 1  order <- StudyOrder(subject)
 2  test <- []
 3  FOR EACH t IN order DO
 4      FOR EACH level IN [EASY, MEDIUM, HARD] DO
 5          FOR EACH id IN topicIndex[t] INTERSECT levelIndex[level] DO
 6              IF practiceFlag[id] AND size(test) < n THEN
 7                  test.append(id)
 8  RETURN test         // easy before hard within each topic
```

4.1.6 Review workflow

Every submitted question waits in a first-in-first-out queue. The administrator takes the question at the front of the queue and either approves or rejects it with a comment. Only an approved question is inserted into the tries and the inverted indexes and becomes available for search and paper generation. Rejected questions go back to the faculty member for correction.

Algorithm 4.10: ReviewNext and Decide

```
ALGORITHM ReviewNext(adminId)
 1  IF reviewQueue.isEmpty() THEN RETURN NULL
 2  id <- reviewQueue.dequeue()          // FIFO
 3  RETURN questionsById.get(id)

ALGORITHM Decide(adminId, id, approve, comment)
 1  q <- questionsById.get(id)
 2  IF approve THEN
 3      q.status <- APPROVED
 4      indexQuestion(q)       // tries + inverted indexes
 5  ELSE
 6      q.status <- REJECTED
 7  reviews.add(Review(id, adminId, approve, comment, now()))
 8  notify(q.createdBy, "Review completed")
 9  persist(q)
```

4.1.7 Administration and undo

Administrative actions are allowed only for users whose role is ADMIN. Every action is pushed on a stack so that the last action can be undone. Approving a faculty account makes it active, and deleting a question removes it from every index so that it no longer appears in searches or papers.

Algorithm 4.11: ApproveUser and DeleteQuestion

```
ALGORITHM ApproveUser(adminId, userId)
 1  IF usersById.get(adminId).role != ADMIN THEN
        RETURN Error("Not authorised")
 2  u <- usersById.get(userId); u.status <- ACTIVE
 3  auditLog.push(Action("APPROVE", adminId, userId, now()))
 4  persist(u)

ALGORITHM DeleteQuestion(adminId, id)
 1  (role check as above)
 2  q <- questionsById.remove(id)
 3  removeFromIndexes(id, q)        // tries, inverted indexes
 4  auditLog.push(Action("DELETE", adminId, q, now()))
 5  persist(q)

ALGORITHM UndoLastAction()
 1  IF auditLog.isEmpty() THEN RETURN
 2  a <- auditLog.pop()
 3  reverse(a)          // e.g. DELETE -> restore question
```

4.1.8 Usage tracking and reports

Every time a question is placed in a paper or a practice test, its usage count is increased. A hash map keeps the counters, and a max-heap over the counters gives the most-used questions for the administrator’s report. This helps in retiring over-used questions and in finding approved questions that have never been used.

Algorithm 4.12: Usage statistics

```
ALGORITHM RecordUsage(id)
 1  usage[id] <- usage.getOrDefault(id, 0) + 1
 2  persist(usageRow(id))

ALGORITHM MostUsed(K)
 1  heap <- new MaxHeap()
 2  FOR EACH (id, c) IN usage DO
 3      heap.push(c, id)
 4  RETURN top K items of heap

ALGORITHM UnusedQuestions()
 1  RETURN approvedSet MINUS usage.keys()
```

4.2 Front-end Logic (UI Rendering and State Management)

The front end is built with HTML5, CSS3 and JavaScript. Every page has a fixed layout with a navigation bar whose menu items depend on the role of the user. The pages are rendered from data returned by the application layer in JSON format. After each request, the JavaScript code updates only the part of the page that has changed, such as the list of search results or the review queue, instead of reloading the whole page.

The state of the interface is kept in a small JavaScript object that stores the logged-in user and role, the current search query and filters, the list of results, the blueprint being edited and the question under review. Whenever the state changes, the corresponding view is drawn again. To avoid sending a request for every key press, the autocomplete box uses a short delay (debouncing) before calling the search service. Forms are validated in the browser before submission and again on the server.

4.3 API Integration

The front end communicates with the application layer through the REST-style endpoints given in Table 4.1. All endpoints other than registration and login require a valid session token, and the role of the user is checked before the operation is carried out.

Table 4.1: Main API endpoints

Method	Endpoint	Purpose
POST	/api/register	Register a new faculty member or student

Method	Endpoint	Purpose
POST	/api/login	Authenticate and create a session
GET / POST	/api/topics	View the syllabus tree or add a topic
POST	/api/topics/{id}/prerequisite	Add a prerequisite after the cycle check
GET / POST	/api/questions	List or add questions
PUT / DELETE	/api/questions/{id}	Edit or delete a question
GET	/api/search?q=&topic=&level=&marks=	Search and filter questions
GET	/api/autocomplete?prefix=	Suggest topics and keywords for a prefix
GET / POST	/api/reviews	Get the next question to review or record a decision
POST	/api/papers	Generate a question paper from a blueprint
GET	/api/practice?subject=&n=	Build a practice test
POST	/api/admin/approve, /block, /undo	Administrative actions

4.4 Persistence and Index Rebuilding

Because the indexes are held in memory, they must be rebuilt whenever the server starts. The start-up routine reads each table from the database once and fills the hash maps, the tries, the inverted indexes, the graph and the heaps. After that, every change is applied to both the database and the in-memory structures.

Algorithm 4.13: BuildIndexes (start-up)

```
ALGORITHM BuildIndexes()
 1  FOR EACH row IN database.users DO
 2      usersById.put(row.id, row)
 3      usersByEmail.put(row.email, row)
 4  FOR EACH row IN database.topics DO
 5      syllabusTree.add(row)
 6  FOR EACH row IN database.prerequisites DO
 7      topicGraph.adj[row.a].add(row.b)
 8  FOR EACH row IN database.questions DO
 9      questionsById.put(row.id, row)
10      IF row.status = APPROVED THEN indexQuestion(row)
11      IF row.status = PENDING THEN
12          reviewQueue.enqueue(row.id)   // ORDER BY created_at
13  FOR EACH row IN database.usage DO
14      usage[row.id] <- row.count
```

The start-up cost is linear in the stored data and is paid once per server start.

5. RESULTS

5.1 Output Screens

This section presents the main screens of the Question Bank Management System web application. Each figure is numbered as ChapterNo.FigureNo and carries a caption, as required by the guidelines.

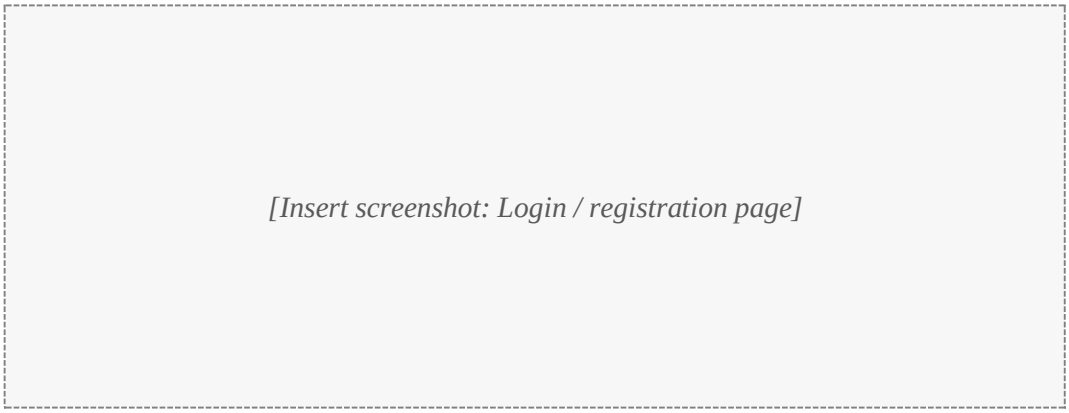


Fig. 5.1 Login and registration page

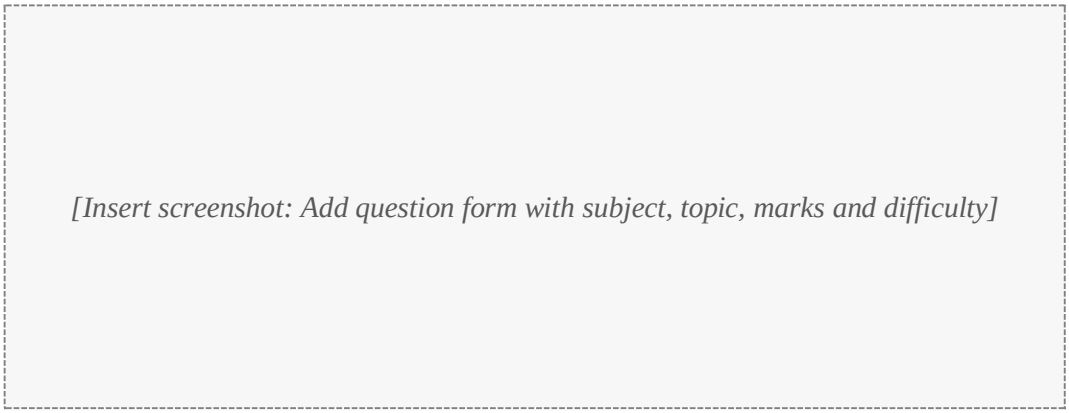


Fig. 5.2 Add question page

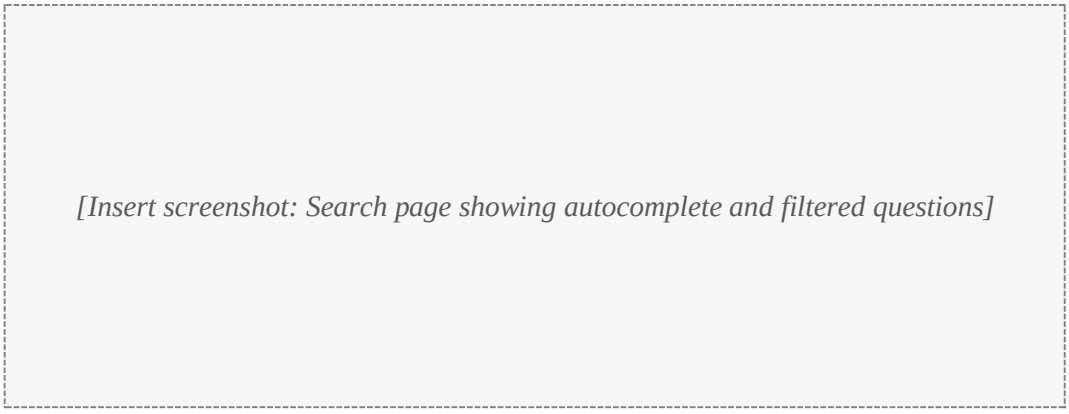


Fig. 5.3 Question search with filters and autocomplete



Fig. 5.4 Review queue and question approval page

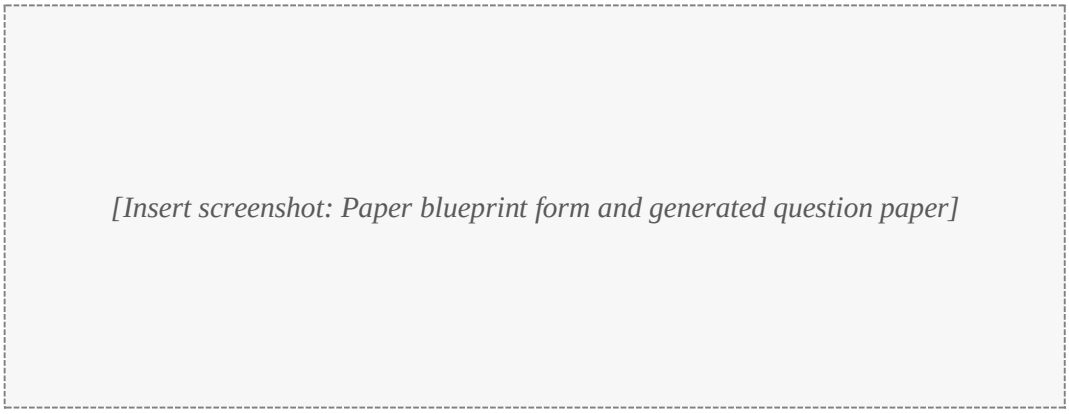


Fig. 5.5 Paper blueprint and generated question paper



Fig. 5.6 Administrator dashboard

The screens show that a faculty member can add and find questions quickly and generate a paper from a blueprint, and that an administrator can review questions and manage users from one dashboard.

5.2 Functional Test Cases

The main functions were checked using the test cases of Table 5.1. For each test case the table gives the input and the behaviour expected from the system.

Table 5.1: Functional test cases and expected results

ID	Test scenario	Input	Expected result
T1	Register with a new e-mail	Valid name, e-mail, password	Account created (student ACTIVE, faculty PENDING)
T2	Register with an existing e-mail	E-mail already stored	Error: “Email already registered”
T3	Login with a wrong password	Valid e-mail, wrong password	Error: “Invalid credentials”
T4	Add an exact duplicate question	Same text as a stored question	Error: “Duplicate question”
T5	Search by topic prefix	“sor”	Questions on topics starting with “sor” are listed
T6	Search with filters	Topic = Stack, difficulty = Medium	Only questions satisfying both filters are shown
T7	Add a prerequisite that forms a cycle	Edge Graphs → Arrays	Error: “Cycle detected”
T8	Review a submitted question	Approve the question at the front of the queue	Status APPROVED; question appears in search
T9	Generate a paper with too few questions	Section needs 5, only 3 approved	Error: “Not enough questions”
T10	Generate a valid paper	Blueprint with units, marks and difficulty	Paper meets the blueprint; usage counts increase
T11	Access an admin page as a student	Student session	Access denied

5.3 Complexity Analysis

Table 5.2 gives the time complexity of the main operations. Here m is the length of the string, n is the number of candidates, K is the number of results required, k is the number of questions required for a section, w is the number of words in a question, and V and E are the numbers of vertices and edges in the topic graph.

Table 5.2: Time complexity of the main operations

Operation	Data structure	Time complexity
Login / look-up by e-mail or id	Hash map	O(1) average
Insert a topic or keyword	Trie	O(m)
Prefix search / autocomplete	Trie	O(m + number of nodes below the prefix)

Operation	Data structure	Time complexity
Apply a filter	Inverted index (hash set)	O(1) look-up; O(min(|A|, |B|)) per intersection
Exact duplicate check	Hash set	O(w) average
Near-duplicate check	Inverted index + Jaccard	O(w) per similar candidate
Top-K ranking of n candidates	Max-heap	O(n + K log n)
Select k least-used questions from n	Min-heap	O(n + k log n)
Study order / cycle check	Queue (Kahn) / DFS	O(V + E)
Order a paper by section and marks	Merge sort	O(k log k)
Review next question	Queue	O(1)
Admin action / undo	Stack	O(1)

5.4 Discussion

The analysis shows that the operations used most often, namely login, duplicate checking, queue handling and filter look-up, run in constant time, while search and paper generation depend only on the length of the query and the number of candidates and not on the total number of questions. Compared with scanning a list of all questions, which would cost O(N) for every search, the trie and the inverted index give the system the ability to remain responsive as the question bank grows. The use of a min-heap for selection avoids sorting all candidates when only a few questions are needed and spreads the use of questions evenly across papers, and the prerequisite graph gives students a sensible order of study.

6. CHALLENGES & LIMITATIONS

6.1 Issues Faced during Development

- **Slow search:** a simple scan over all questions for each key press made autocomplete slow as the number of questions increased.
- **Inconsistent indexes:** after a question was edited, old keywords remained in the indexes, so the question appeared in wrong search results.
- **Duplicate questions:** the same question was entered several times with small changes in wording, so exact comparison alone was not enough.
- **Repeated questions in papers:** choosing questions only by the blueprint made the same questions appear paper after paper.
- **Cyclic prerequisites:** a wrong prerequisite could make two topics depend on each other, which made a study order impossible.
- **Unreviewed questions:** questions with wrong answers or poor wording could enter a paper before they were checked.

6.2 Solutions Applied

- A trie and inverted indexes replaced the list scan, so that search time no longer depends on the total number of questions.
- A removeFromIndexes step was added before every question update, so that old entries are deleted before the new ones are inserted.
- A Jaccard similarity check on the words of a question, limited to questions that share words, was added to detect near-duplicates.
- A min-heap keyed by usage count, together with the exclusion of recently used questions, spreads the use of questions across papers.
- A depth-first cycle check is made before every prerequisite is added.
- A review queue was introduced, and only approved questions are indexed and used in papers.

6.3 Limitations of the System

- The indexes are held in the memory of a single server, so very large

question banks would need a distributed index or a search engine.

- The difficulty and cognitive level of a question are assigned by the faculty member; the system does not verify them automatically.
- The duplicate check compares words only, so it cannot detect questions that have the same meaning but use different words.
- Practice tests are evaluated automatically only for objective questions; descriptive answers are not graded.
- The system is a responsive web application; there is no native mobile application and no proctored online examination.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

This project designed and developed the Question Bank Management System, a web-based platform that stores, reviews and reuses examination questions. It provides syllabus management, question entry with duplicate detection, search with filters and autocomplete, a review and approval workflow, automatic generation of question papers from a blueprint, topic-wise practice tests and an administrative module. By saving the time of faculty, avoiding repeated questions and keeping the quality of questions under review, the system helps the institution maintain an organised, reusable and fair question repository.

From the point of view of the course, the project shows how the right data structures make a real application efficient: hash maps for constant-time look-ups, a trie for prefix search, an inverted index for filters, a tree and a graph for the syllabus, heaps for selection and ranking, and queues and stacks for the review workflow and administration. The main limitations are the single-server in-memory indexes and the word-based duplicate check.

7.2 Future Work

- A mobile application with offline access to practice tests.
- Detection of duplicate questions by meaning, using text embeddings.
- Predicting the difficulty of a question from the results of students using machine learning.
- Online proctored examinations with automatic evaluation of objective answers.
- Import of questions from Word and PDF files, and export of papers in the university format.
- Analytics dashboards on question usage, difficulty and student performance.
- A distributed search index for very large question banks.

7.3 Skills Learned during Development

- Choosing and applying suitable data structures and algorithms to a real problem and analysing their complexity.
- Writing clear pseudocode and converting it into working code.
- Designing a layered architecture and REST-style services.
- Front-end development with HTML, CSS and JavaScript, including state management.
- Database design, version control with Git, testing and teamwork.
- Technical report writing and presentation.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[2] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2013.

[3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011.

[4] M. E. J. Newman, *Networks: An Introduction*. Oxford, U.K.: Oxford University Press, 2010.

[5] E. Fredkin, "Trie memory," *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, 1960.

[6] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.

[7] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[8] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.

[9] Python Software Foundation, "Python 3 documentation." [Online]. Available: https://docs.python.org/3/

[10] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com/

[11] MDN Web Docs, "Web technology for developers." [Online]. Available: https://developer.mozilla.org/

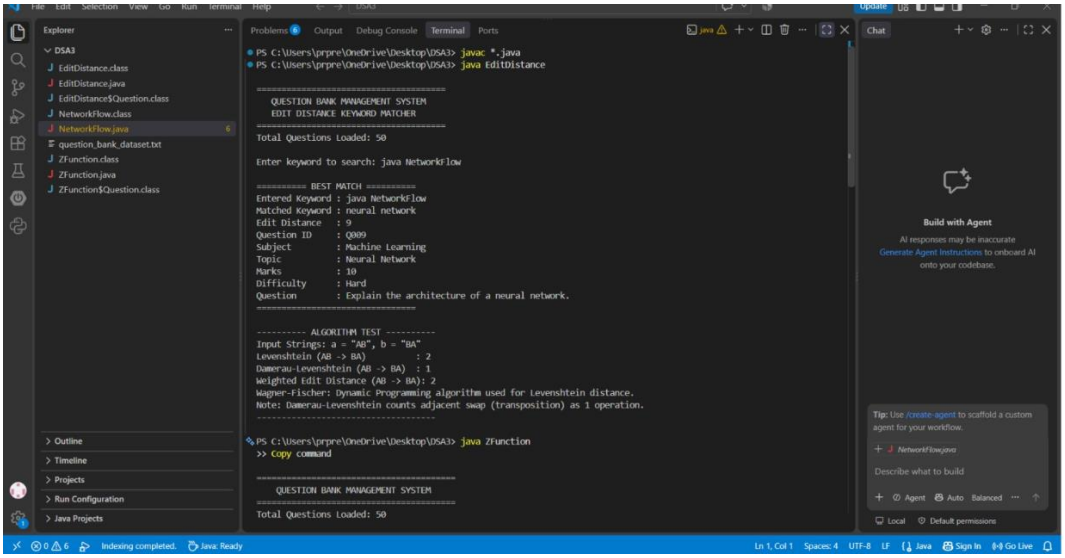
[12] SQLite Consortium, "SQLite documentation." [Online]. Available: https://www.sqlite.org/docs.html

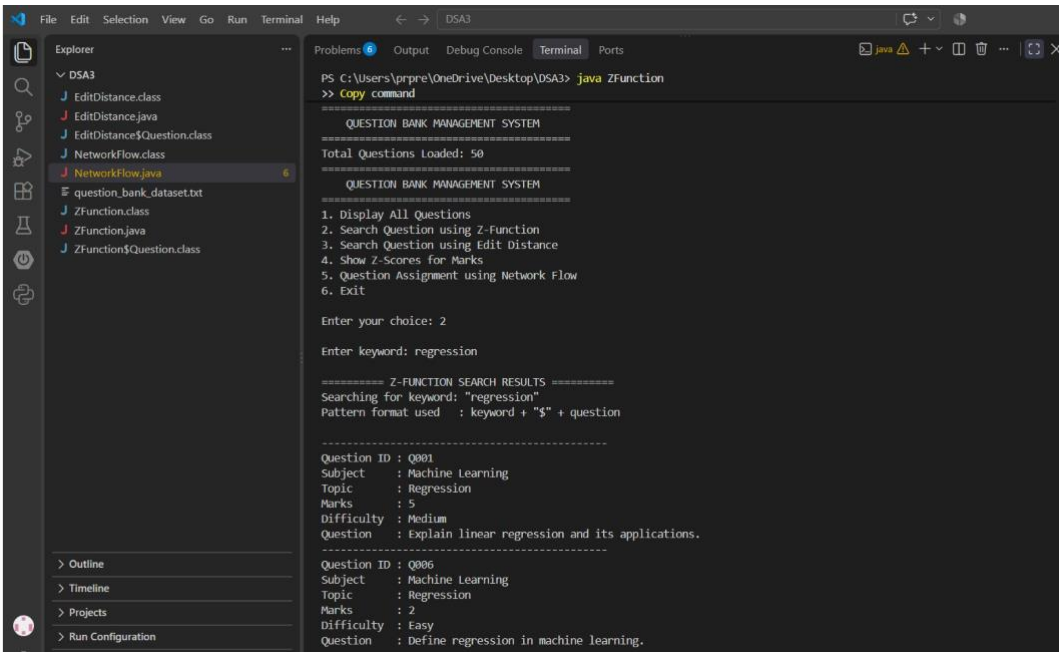
[13] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.

[14] B. S. Bloom, Ed., *Taxonomy of Educational Objectives, Handbook I: Cognitive Domain*. New York, NY, USA: David McKay, 1956.

[15] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, Salerno, Italy, 1997, pp. 21–29.

EXPECTED OUTPUT SCREENSHOTS





```
PS C:\Users\prpre\OneDrive\Desktop\DSA3> java ZFunction
>> Copy command
--------------------------------------------
Question ID : Q031
Subject     : Machine Learning
Topic       : Regression
Marks       : 10
Difficulty  : Hard
Question    : Explain the difference between simple and multiple linear regression.
--------------------------------------------
Question ID : Q032
Subject     : Machine Learning
Topic       : Classification
Marks       : 5
Difficulty  : Medium
Question    : Explain logistic regression and its applications.
--------------------------------------------
Total Matching Questions: 4
============================================

====================================
    QUESTION BANK MANAGEMENT SYSTEM
====================================
1. Display All Questions
2. Search Question using Z-Function
3. Search Question using Edit Distance
4. Show Z-Scores for Marks
5. Question Assignment using Network Flow
6. Exit

Enter your choice:
```

